

[CS230] Convolutional Neural Networks: YOLO Algorithm (Object Detection)

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-8. Deep Learning Strategy & Architecture - *Completed*

Chapter 9. Convolutional Neural Networks (Current Part)

- 9.1-9.6 CNN Basics & Sliding Windows - *Completed*
- **9.7 YOLO Algorithm (You Only Look Once)**
 - * The Grid System & Bounding Box Regression
 - * IoU (Intersection over Union) Metric
 - * Non-max Suppression (Removing Duplicates)
 - * Anchor Boxes (Handling Overlap)

Chapter 10. Sequence Models (RNN) - *Next Part*

지난 시간 복습 및 연결

지난 시간에 배운 슬라이딩 윈도우는 합성곱 구현으로 속도는 빨라졌지만, 여전히 "박스 위치가 부정확하다"는 한계가 있었습니다. 윈도우가 고정된 간격으로만 움직이기 때문입니다. "객체의 중심을 찾고, 그 중심을 기준으로 박스 크기를 예측하면 어떨까?" 이 아이디어로 탄생한 것이 **YOLO**입니다. 이름처럼 이미지를 단 한 번만 보고 (Look Once), 모든 객체의 위치와 종류를 동시에 찾아내는 혁신적인 알고리즘입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 실시간 객체 탐지의 표준인 YOLO 알고리즘의 원리를 완벽히 이해합니다.

- **그리드:** 이미지를 격자로 나누고, 객체 중심점이 속한 셀이 책임을 지는 구조를 배웁니다.
- **IoU:** 두 박스가 얼마나 겹치는지를 측정하는 평가 지표를 수학적으로 정의합니다.
- **NMS:** 중복된 박스를 제거하는 비최대 억제 (Non-max Suppression) 알고리즘을 익힙니다.
- **앵커:** 겹친 물체를 분리하는 앵커 박스 (Anchor Box) 개념을 파악합니다.

2 Essential Terminology

용어	약어	설명
Grid Cell	-	이미지를 $S \times S$ 로 나눈 작은 구역.
IoU	Intersection over Union	교집합 영역 / 합집합 영역. (일치도)
NMS	Non-max Suppression	가장 확실한 박스 하나만 남기고 나머지는 지움.
Anchor Box	-	미리 정의된 박스 모양. (길쭉한 사람, 넓은 차 등)

3 Core Concepts: 단 한 번의 추론

3.1 1. Bounding Box Predictions (그리드 시스템)

YOLO는 이미지를 $S \times S$ 그리드(보통 19×19)로 나눕니다. 각 셀은 다음 벡터 y 를 예측합니다.

$$y = [p_c, b_x, b_y, b_h, b_w, c_1, c_2, \dots]^T$$

- p_c : 객체가 있을 확률 (Confidence).
- b_x, b_y : 박스 중심 좌표 (셀 내 상대 위치, 0 1).
- b_h, b_w : 박스 높이/너비 (전체 이미지 대비 비율).
- c_i : 클래스 확률 (차, 사람 등).

3.2 2. IoU (Intersection over Union)

모델이 예측한 박스가 정답과 얼마나 비슷한지 평가하는 척도입니다.

□ IoU 수식 (수학적 정의)

$$\text{IoU} = \frac{\text{교집합 영역 (Intersection)}}{\text{합집합 영역 (Union)}}$$

- 보통 $\text{IoU} \geq 0.5$ 이면 "올바른 탐지"로 간주합니다.
- 1이면 완벽하게 일치, 0이면 전혀 겹치지 않음.

3.3 3. Anchor Boxes (겹친 물체 해결)

한 셀의 중심에 사람과 차가 겹쳐 있다면? 기존 벡터로는 하나만 예측 가능합니다. 이를 위해 미리 정의된 모양(앵커)을 사용합니다.

$$y = [\text{Anchor 1}, \text{Anchor 2}]$$

- **Anchor 1 (세로로 긴):** 사람 담당.
- **Anchor 2 (가로로 넓은):** 자동차 담당.

4 Deep Dive: Non-max Suppression (NMS)

YOLO는 하나의 객체에 대해 여러 셀이 "내가 찾았다!"며 박스를 칠 수 있습니다. 중복을 제거해야 합니다.

1. **Filter:** $p_c < 0.6$ 인 박스(확신 없는 것)는 모두 버립니다.
2. **Select:** 남은 박스 중 p_c 가 가장 높은 것을 선택합니다 (Best Box).
3. **Suppress:** 선택된 박스와 $\text{IoU} \geq 0.5$ 인(많이 겹친) 다른 박스들은 "같은 물체를 중복 탐지한 것"으로 보고 지웁니다.
4. **Repeat:** 박스가 다 정리될 때까지 반복합니다.

5 Implementation: IoU Calculation

IoU 계산은 객체 탐지 성능 평가와 NMS 구현의 핵심입니다.

```
1 def calculate_iou(box1, box2):
2     """
3     box: (x1, y1, x2, y2) 좌표 좌상단(, 우하단)
4     """
5     (b1_x1, b1_y1, b1_x2, b1_y2) = box1
6     (b2_x1, b2_y1, b2_x2, b2_y2) = box2
7
8     # 1. 교집합(Intersection) 좌표 계산
9     xi1 = max(b1_x1, b2_x1)
10    yi1 = max(b1_y1, b2_y1)
11    xi2 = min(b1_x2, b2_x2)
12    yi2 = min(b1_y2, b2_y2)
13
14    # 넓이 음수면( 0)
15    inter_area = max(0, xi2 - xi1) * max(0, yi2 - yi1)
16
17    # 2. 합집합(Union) 넓이 계산
18    b1_area = (b1_x2 - b1_x1) * (b1_y2 - b1_y1)
19    b2_area = (b2_x2 - b2_x1) * (b2_y2 - b2_y1)
20    union_area = b1_area + b2_area - inter_area
21
22    # 3. IoU
23    return inter_area / (union_area + 1e-6)
24
25 # --- 테스트 ---
26 if __name__ == "__main__":
27     box_a = (1, 1, 3, 3) # 면적 4
28     box_b = (2, 2, 4, 4) # 면적 4, 교집합 1
29     # 합집합 = 4 + 4 - 1 = 7
30     # IoU = 1/7 = 0.1428...
31
32     print(f"IoU: {calculate_iou(box_a, box_b):.4f}")
```

Listing 1: IoU Calculation Function

6 FAQ & Pitfalls

좌표계 주의 (오해 방지 가이드)

YOLO의 출력 b_x, b_y 는 그리드 셀 내부에서의 상대 위치(0 1)입니다. 실제 이미지 위에 박스를 그리려면, 셀의 위치(인덱스)를 더하고 이미지 크기를 곱해주는 변환 과정이 필요합니다.

Q. 앵커 박스 크기는 어떻게 정하나요?

A. 보통 훈련 데이터에 있는 객체들의 실제 박스 크기를 모아서 **K-Means** 클러스터링을 돌립니다. 가장 빈번하게 등장하는 대표적인 모양 5 9개를 선정하여 사용합니다 (YOLO v2부터 적용).

다음 단계 (Next Step)

이것으로 컴퓨터 비전(CNN) 파트를 마칩니다. 이제 여러분은 정지된 이미지에서 사물을 분류하고 위치까지 찾아내는 기술을 습득했습니다.

하지만 세상은 멈춰 있지 않습니다. 유튜브 영상, 음성 인식, 주가 예측, 번역 등은 **시간의 흐름(Sequence)**이 있는 데이터입니다. 다음 시간부터는 [Part 5. Sequence Models]의 세계로 떠납니다. 시계열 데이터를 처리하는 가장 기본적인 신경망, **RNN (Recurrent Neural Networks)**에 대해 알아보겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **YOLO**: 그리드 셀마다 바운딩 박스를 회귀(Regression)로 직접 예측한다.
2. **IoU**: 교집합/합집합. 박스 위치 정확도의 척도이자 NMS의 기준.
3. **NMS**: 중복된 박스를 제거하여 객체당 하나의 박스만 남긴다.
4. **Anchor**: 다양한 비율의 객체를 잡기 위해 미리 정의된 박스 모양을 쓴다.